■ is_prime function (Python)

Language: **Python** | Prepared by **Aman Samriya** | 2026-05-13

■ Original Code

```
def is_prime(n):
    if n < 2:
        return False
    for i in range(2, int(n**0.5)+1):
        if n % i == 0:
            return False
    return True
```

■ What this code does (Layman)

Determines whether a given integer is a prime number. It returns True for primes and False for non-primes by testing divisibility up to the square root of the number.

■ Step-by-Step Explanation

- Check if n is less than 2; if so, it is not prime, so return False.
- Compute the integer range of potential divisors from 2 up to floor(sqrt(n)).
- For each candidate divisor i in that range, if n is divisible by i (n % i == 0), then n is composite, so return False.
- If no divisors are found in the loop, return True because n is prime.

■ Real-World Analogy

Like checking if a new key is unique by trying it in every lock up to a point where trying more would only repeat previous checks; you only try locks up to the square-root boundary because beyond that you would be re-checking complementary pairs.

■ Inputs & Outputs

Input	Output
is_prime(2)	True
is_prime(15)	False
is_prime(17)	True
is_prime(1)	False

■■ What it Delivers

A simple, readable primality test suitable for small to moderate integer inputs; useful in scripts, teaching, or quick checks where performance for very large numbers is not required.

■ Edge Cases & Assumptions

• n < 2 (returns False) — covers 0 and 1 and negatives, but negatives conceptually aren't prime.

• Non-integer inputs (e.g., 3.5) — the function assumes integers and may behave unpredictably or raise an error elsewhere.

• Very large n — performance degrades (O(sqrt(n))).

• Floating-point precision when computing n**0.5 for extremely large integers (use math.isqrt to avoid floats).

■ Strengths

• Very simple and easy to understand implementation.

• Uses the sqrt(n) optimization (only checks divisors up to sqrt(n)), which is standard and effective for small inputs.

• Correct for integer inputs >= 2.

■■ Weaknesses

• No input validation or type checking — non-integers are not handled explicitly.

• Uses floating-point sqrt which can introduce edge-case imprecision for very large integers.

• Inefficient for large numbers compared to probabilistic or advanced deterministic tests (e.g., Miller–Rabin, AKS).

• Can be sped up by skipping even numbers after checking for divisibility by 2.

■■ Suggested Improvements

• Add input validation: ensure n is an integer and handle or reject non-integer values.

• Use math.isqrt(n) to compute integer square root and avoid floating-point issues: limit = math.isqrt(n).

• Handle small primes explicitly (check n == 2) and skip even divisors: if n % 2 == 0: return n == 2; then loop range(3, limit+1, 2).

• For very large inputs, use a faster probabilistic test such as Miller–Rabin (with a few bases) or a deterministic test for required ranges.

• Add docstring and unit tests; consider caching results for repeated queries.

■ Final Summary

A concise, correct trial-division primality check suitable for small numbers; for robust, large-scale use, add validation and replace the algorithm with a faster method.